

Bicol University College of Science
Department of Information Technology

IT106 - Application Development in Emerging Technologies
2nd Semester 2025-2026

Laboratory Activity:
Automated Testing of Final Project Functionalities (Selected)
PlantTrade: A Web-based E-commerce Platform for Marga's Garden
Plant Nursery Business
Group Journal

Submitted By:
Janice B. Barbacena
Vince Mathew O. Temajo
Carl Justine O. Tuazon
Cayla C. Villanueva

BS Information Technology - 3rd Year

Dr. Jayvee Christopher N. Vibar
Faculty-in-charge

May 20, 2026

I. Testing Details Per Member

Member	Assigned Feature	Framework Used
Janice B.	Login and Authentication	Selenium
Vince T.	Customer Cart	Selenium
Justine T.	Category Management	Selenium
Cayla V.	Admin Transaction Management	Selenium

II. Test Scenario Documentation

Signup Functionality Testing

Assigned Member: Janice Barbacena

1. **Scenario:** Sign up succeeds with valid username, email, password, and confirm password.
- **Functionality Tested:** User registration
 - **Objective of the Test:** Verify that the system creates a new account when all signup fields contain valid values.
 - **Steps/Procedure:**

1. Open the signup page.

2. Submit a signup request with a unique username, unique email, valid password, and matching confirm password.

3. Validate the API response.
 - **Test Data/Input:** username=signup_<unique>, email=signup_<unique>@example.com, password=abcABC123!@#, confirmPassword=abcABC123!@#
 - **Expected Result:** The system creates the account successfully
 - **Actual Result:** The system creates the account successfully
 - **Status (Passed/Failed):** Passed
2. **Scenario:** Sign up fails when email already exists.
- **Functionality Tested:** Duplicate email validation in signup
 - **Objective of the Test:** Verify that the system rejects account creation when the email address is already registered.
 - **Steps/Procedure:**

1. Create an initial account with a unique email.
 2. Clear the session.
 3. Attempt another signup using the same email and a different username.
 4. Validate the API response.
- **Test Data/Input:** Existing email with a different username
 - **Expected Result:** The system rejects the request and an already-registered message.
 - **Actual Result:** The system rejects the request and an already-registered message.
 - **Status (Passed/Failed):** Passed
3. **Scenario:** Sign up fails when password and confirm password do not match.
- **Functionality Tested:** Password confirmation validation in signup
 - **Objective of the Test:** Verify that the system rejects signup when password and confirm password are different.
 - **Steps/Procedure:**
 1. Open the signup page.
 2. Submit signup data with different values for password and confirm password.
 3. Validate the API response.
 - **Test Data/Input:** password=abcABC123!@#, confirmPassword=differentPassword123!
 - **Expected Result:** The system rejects the request and a password-mismatch message.
 - **Actual Result:** The system rejects the request and a password-mismatch message.
 - **Status (Passed/Failed):** Passed

```
[RUN ] Sign up succeeds with valid username, email, password, and confirm password.
[PASS] Sign up succeeds with valid username, email, password, and confirm password.

[RUN ] Sign up fails when password and confirm password do not match.
[PASS] Sign up fails when password and confirm password do not match.

[RUN ] Sign up fails when email already exists.
[PASS] Sign up fails when email already exists.

Completed 3 signup Selenium tests.
All signup Selenium tests passed.
```

Figure 1. Screenshot of actual test run

Issues Encountered:

One issue encountered during testing was that the successful signup scenario occasionally failed because the email used in a previous run was still stored in the database. At first this looked like a bug in the signup feature, but after checking the

records, it turned out to be a test-data conflict caused by rerunning the script with old inputs.

Improvements Made After Testing:

One improvement made after testing was changing the script to generate unique usernames and emails automatically for the valid signup case. This reduced repeated-data conflicts and made the automation more reliable when the test was run several times in a row.

Lessons Learned from Automation Testing:

One lesson learned from automation testing is that a failing test result does not always mean the feature itself is broken. Sometimes the real cause is the environment, old database data so it is important to verify the root cause before concluding that there is a system defect.

Customer Cart

Assigned Member: Vince Mathew Temajo

1. **Scenario:** Duplicate cart items merge by inventory/variation instead of duplicating incorrectly.
 - **Functionality Tested:** Customer cart item merge behavior
 - **Objective of the Test:** Verify that when the same inventory variation is added more than once, the system merges the entries into one cart item instead of creating duplicates.
 - **Steps/Procedure:**
 1. Create a new user account to start an authenticated session.
 2. Retrieve one valid product variation from the live catalog.
 3. Save two cart entries using the same inventoryId and variation, but with different quantities.
 4. Fetch the remote cart contents.
 5. Check whether only one merged item remains with the combined quantity.
 - **Test Data/Input:** Same live inventory item submitted twice with quantities 2 and 3
 - **Expected Result:** The cart stores only one item for that inventory/variation, and the quantity becomes 5.

- **Actual Result:** The cart stores only one item for that inventory/variation, and the quantity becomes 5.
- **Status (Passed/Failed):** Passed

2. **Scenario:** Zero or negative quantity items are discarded.

- **Functionality Tested:** Customer cart quantity validation
- **Objective of the Test:** Verify that items with invalid non-positive quantities are ignored and not stored in the remote cart.
- **Steps/Procedure:**
 1. Create a new authenticated user session.
 2. Retrieve one valid product variation from the live catalog.
 3. Save cart items using the same valid item but with quantities 0 and -2.
 4. Fetch the remote cart contents.
 5. Check whether the cart remains empty.
- **Test Data/Input:** Same live inventory item with quantities 0 and -2
- **Expected Result:** The invalid items are discarded and no cart item is saved.
- **Actual Result:** The invalid items are discarded and no cart item is saved.
- **Status (Passed/Failed):** Passed

3. **Scenario:** Quantity updates persist remotely for authenticated users.

- **Functionality Tested:** Customer cart remote persistence
- **Objective of the Test:** Verify that when an authenticated user updates a cart quantity, the new quantity is saved and remains correct when the cart is fetched again.
- **Steps/Procedure:**
 1. Create a new authenticated user session.
 2. Retrieve one valid product variation from the live catalog.
 3. Save the item to the cart with quantity 1.
 4. Fetch the cart and confirm the initial quantity.
 5. Save the same item again with quantity 4 as the updated state.
 6. Fetch the cart again and verify the new quantity.
- **Test Data/Input:** Same live inventory item first saved with quantity 1, then updated to quantity 4
- **Expected Result:** The remote cart keeps one item and its stored quantity changes from 1 to 4.
- **Actual Result:** The remote cart keeps one item and its stored quantity changes from 1 to 4.
- **Status (Passed/Failed):** Passed

```
[RUN ] Duplicate cart items merge by inventory/variation instead of duplicating incorrectly.  
[PASS] Duplicate cart items merge by inventory/variation instead of duplicating incorrectly.  
  
[RUN ] Zero or negative quantity items are discarded.  
[PASS] Zero or negative quantity items are discarded.  
  
[RUN ] Quantity updates persist remotely for authenticated users.  
[PASS] Quantity updates persist remotely for authenticated users.
```

Figure 2. Screenshot of actual test run

Issues encountered:

During testing, some cart scenarios became unreliable when hardcoded sample product data did not exactly match the current catalog records in the database. If the inventoryID or variation no longer existed or had changed, the cart test could fail even when the cart logic itself was working correctly.

Improvements made after testing:

The script was improved by pulling a real inventory combination from the catalog API so cart requests use valid inventoryID values from the actual database instead of hardcoded sample items.

Lesson learned:

Cart automation is more reliable when it uses live and valid product data instead of fixed assumptions. Tests that depend on inventory, variations, or database records should stay aligned with the current system data to avoid false failures.

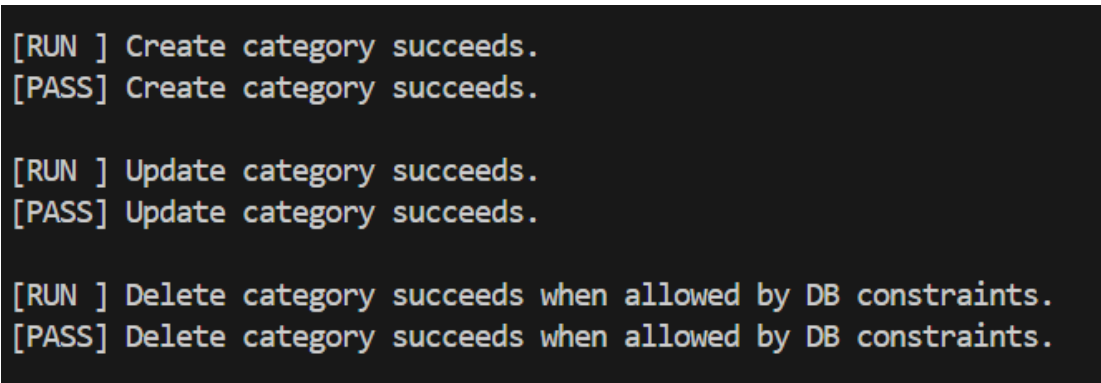
Category Management

Assigned Member: Carl Justine Tuazon

1. **Scenario:** Create category succeeds.
 - **Functionality Tested:** Admin catalog category creation
 - **Objective of the Test:** Verify that an admin user can create a new category successfully through the admin catalog API.
 - **Steps/Procedure:**
 1. Prepare the admin account credentials.
 2. Log in as admin using the real login API.

3. Submit a category save request with a new category name, description, and profit margin.
 4. Check the returned catalog data for the new category entry.
 - **Test Data/Input:** Unique category name such as Created Category <timestamp>, description text, profitMargin=15
 - **Expected Result:** The API returns, and the new category appears in the active category list.
 - **Actual Result:** The API returns, and the new category appears in the active category list.
 - **Status (Passed/Failed):** Passed
2. **Scenario:** Update category succeeds.
- **Functionality Tested:** Admin catalog category update
 - **Objective of the Test:** Verify that an existing category can be updated successfully by an admin user.
 - **Steps/Procedure:**
 1. Log in as admin.
 2. Create a temporary category for safe test editing.
 3. Submit an update request using the created category ID with a new name, new description, and updated profit margin.
 4. Check the returned catalog data for the updated values.
 - **Test Data/Input:** Temporary category updated to a new name ending in updated, new description, profitMargin=22
 - **Expected Result:** The API returns, and the category reflects the updated name, description, and profit margin.
 - **Actual Result:** The API returns, and the category reflects the updated name, description, and profit margin.
 - **Status (Passed/Failed):** Passed.
3. **Scenario:** Delete category succeeds when allowed by DB constraints.
- **Functionality Tested:** Admin catalog category deletion
 - **Objective of the Test:** Verify that a category can be deleted successfully when it is safe to remove under the database rules.
 - **Steps/Procedure:**
 1. Log in as admin.
 2. Create a temporary category specifically for deletion.
 3. Submit a delete request using that category's ID.
 4. Check the returned category list.

5. Fetch the catalog again to confirm the category no longer appears among active categories.
- **Test Data/Input:** Temporary category created during the same test run
 - **Expected Result:** The API returns success=true, and the deleted category is no longer present in the active admin catalog results.
 - **Actual Result:** The API returns success=true, and the deleted category is no longer present in the active admin catalog results.
 - **Status (Passed/Failed):** Passed



```
[RUN ] Create category succeeds.  
[PASS] Create category succeeds.  
  
[RUN ] Update category succeeds.  
[PASS] Update category succeeds.  
  
[RUN ] Delete category succeeds when allowed by DB constraints.  
[PASS] Delete category succeeds when allowed by DB constraints.
```

Figure 3. Screenshot of actual test run

Issues encountered:

I accidentally changed a live catalog data while trying the admin category management scenarios. During an early test attempt, an existing category was edited instead of a temporary one, which affected actual records used by the system. Extra care was needed to isolate the test data and avoid unintended changes to the live catalog.

Improvements made after testing:

The automation was made safer by creating temporary categories for update and delete actions instead of editing permanent catalog records directly.

Lessons learned from automation testing:

It is safer for Admin CRUD automation when each test creates its own disposable data. That keeps the results repeatable and avoids interfering with existing catalog content.

Admin Transactions Management

Assigned Member: Cayla Villanueva

1. **Scenario:** Edit transaction updates customer name.

- **Functionality Tested:** Admin transaction edit
- **Objective of the Test:** Verify that an admin can edit a transaction and update the customer name successfully.
- **Steps/Procedure:**
 1. Create a temporary customer account.
 2. Complete the customer profile and place a real order.
 3. Log out from the customer session.
 4. Log in as admin.
 5. Open the admin transactions data through the API.
 6. Edit the transaction and change the customer name.
 7. Check the returned transaction list for the updated name.
- **Test Data/Input:** Temporary customer order with updated name Updated Customer Name
- **Expected Result:** The edited transaction returns successfully and the customer name is updated in the admin transactions data.
- **Actual Result:** The edited transaction returns successfully and the customer name is updated in the admin transactions data.
- **Status (Passed/Failed):** Passed

2. **Scenario:** Update status changes delivery status in DB.

- **Functionality Tested:** Admin transaction status update
- **Objective of the Test:** Verify that an admin can update the delivery status of a transaction and that the saved value is reflected when the transaction is loaded again.
- **Steps/Procedure:**
 1. Create a temporary customer account.
 2. Complete the customer profile and place a real order.
 3. Log out from the customer session.
 4. Log in as admin.
 5. Update the transaction status using the admin transactions API.
 6. Fetch the same transaction again from the admin transactions list.
 7. Confirm that the updated delivery status is still present.
- **Test Data/Input:** Temporary order status changed to Out for Delivery
- **Expected Result:** The transaction status update succeeds and the transaction shows Out for Delivery when reloaded from the API.

- **Actual Result:** The transaction status update succeeds and the transaction shows Out for Delivery when reloaded from the API.
- **Status (Passed/Failed):** Passed

3. **Scenario:** Purchase history shows invoice number, order date, delivery date, total, items.

- **Functionality Tested:** Customer purchase history display data
- **Objective of the Test:** Verify that a created order appears in purchase history with the essential fields needed for transaction review.
- **Steps/Procedure:**
 1. Create a temporary customer account.
 2. Complete the profile and place a real order.
 3. Fetch the customer purchase history from the orders API.
 4. Find the created order by its database ID.
 5. Check that invoice number, order date, delivery date, total amount, and items are present.
- **Test Data/Input:** Temporary customer order created from a live catalog item.
- **Expected Result:** Purchase history contains a non-empty invoice number, order date, delivery date, total amount greater than zero, and at least one order item.
- **Actual Result:** Purchase history contains a non-empty invoice number, order date, delivery date, total amount greater than zero, and at least one order item.
- **Status (Passed/Failed):** Passed

```
[RUN ] Edit transaction updates customer name.
[PASS] Edit transaction updates customer name.

[RUN ] Update status changes delivery status in DB.
[PASS] Update status changes delivery status in DB.

[RUN ] Purchase history shows invoice number, order date, delivery date, total, item
s.
[PASS] Purchase history shows invoice number, order date, delivery date, total, item
s.
```

Figure 4. Screenshot of actual test run

Issues encountered:

Admin transaction features depend on real order data, so the test could not start directly from the admin side. A temporary customer order had to be created first before edit and delete actions could be tested properly.

Improvements made after testing:

The automation was made more reliable by generating a temporary customer account and order for each scenario, which avoids depending on old seeded transactions.

Lessons learned from automation testing

Transaction-management tests are strongest when they validate both API responses and underlying stored data, because admin features often look correct in the UI even when backend cleanup behavior is incomplete.